

基于YOLOv8的智能车载视觉系统设计与实现

摘要

为满足高级驾驶辅助系统（ADAS）对多任务感知的高实时性与高准确性需求，本文开发了一套集成目标检测、车道线识别、单目测距及3D可视化展示的智能车载视觉系统。系统核心采用轻量化深度学习模型YOLOv8进行前方多类交通参与者（车辆、行人及交通标志）的实时定位与分类；同时结合HSL颜色空间分割和Canny边缘检测等传统算法实现车道线的鲁棒提取；在此基础上，利用单目相机针孔成像模型并结合特定目标的几何先验知识，完成横纵向距离估计；最后通过3D立方体投影算法将环境感知结果直观呈现在图像中。测试结果表明，该系统在主流PC端运行帧率稳定，对典型道路环境具有良好的动态感知能力，能有效支撑碰撞预警等辅助驾驶功能。

关键词：YOLOv8；目标检测；车道线检测；单目测距；高级驾驶辅助系统

1 绪论

1.1 研究背景与意义

智能驾驶技术作为现代汽车工业和信息技术融合的前沿方向，近年来发展迅猛。高级驾驶辅助系统（ADAS）作为实现完全自动驾驶的过渡阶段和核心基础，其技术关键在于如何实现快速、准确的环境感知。通过传感器采集环境数据，系统能为车辆提供前向碰撞预警（FCW）、车道偏离预警（LDW）等主动安全功能，对降低交通事故率有重大意义。相较于红外、激光雷达等高成本传感器，车载视觉系统（摄像头）由于其信息维度丰富、安装便捷、性价比高，目前仍是车载感知领域的研究热点与主流方案。

传统的车载视觉算法多依赖于手工提取的图像特征（如HOG、SIFT等），这类方法在光照突变、复杂路况下的泛化能力较差。近年来，卷积神经网络（CNN）在计算机视觉领域展现出极强的特征表征能力。其中，YOLO系列单阶段目标检测算法凭借其端到端（End-to-End）结构，在速度与精度之间取得了较好的平衡。作为该系列的较新演进版，YOLOv8在特征融合网络结构及损失函数上均做了优化，为满足高帧率的车载感知任务提供了理想的底座。

与此同时，仅获取2D边界框并不能完全满足车辆主动安全决策的需求，系统还需同时感知车道约束并获取目标的三维相对位置。车道线识别能够限制安全行驶区域，而可靠的测距信息则是激活紧急制动（AEB）的前提。虽然双目视觉或雷达可提供直接的深度信息，但其算法复杂度高或硬件成本昂贵。利用单目相机结合目标几何先验进行距离估计，是一种低成本且高效的工业替代路径。

针对上述实际需求，本文搭建了一个多功能融合的车载视觉感知原型系统。通过结合YOLOv8目标检测框架与传统计算机视觉算子，实现了车道线提取、目标实时定位与单目空间测距，并采用3D立体投影技术对感知结果进行直观展示。本项工作既为多任务车载视觉感知提供了一种模块化融合框架，也为低成本主动安全系统的落地应用提供了技术方案参考。

1.2 国内外研究现状

多任务感知一直是车载视觉研究的重要课题，国内外学者在目标识别、车道线提取及单目测距等方向积累了丰富的成果。

目标检测领域经历了从传统滑窗搜索向深度学习的演进。基于R-CNN系列的双阶段（Two-stage）方法通过区域提议网络提升了检测准确率，但多步骤的级联架构导致计算成本偏高，难以直接部署于3D实时感知硬件中。Joseph Redmon等提出的YOLO（You Only Look Once）框架将定位与分类统一转化为一个回归问题，奠定了实时目标检测的基础。随后的YOLOv3、YOLOv4、YOLOv5等版本引入了残差结构、路径聚合网络（PAN）及多元数据增强策略，使其在工业界得到了广泛部署。Ultralytics团队在2023年推出的YOLOv8更进一步，采用了解耦头（Decoupled Head）设计并取消了传统的锚框（Anchor-Free）机制，大幅减少了超参数对数据集的依赖，显著提升了针对长尾分布及小目标的抓取精度。

在车道线检测方面，现有技术主要分为图像处理和深度学习两大阵营。早期方法主要在时空域内通过边缘算子提取车道边缘，再配合霍夫变换（Hough Transform）或多项式拟合输出车道走向。这类方法运算速度快，但严重受限于极端光照（如逆光、暗夜）或路面阴影的干扰。近年来，基于语义分割（如SCNN、LaneNet）的方法虽然增强了在复杂环境下的特征挖掘能力，但这类网络往往参数量庞大，标注成本极高，且在边缘端设备上的推理延迟难以忽视。因此，利用特定的颜色空间（如HSL）配合经典的几何滑动窗口进行鲁棒特征提取，并在帧间引入平滑滤波，仍是一种在计算效率和鲁棒性之间极佳的折中方案。

单目距离估计作为经典的逆问题，由于缺乏直接的深度投影信息，早期多基于逆透视变换（IPM）假定路面为理想平面，通过像素坐标轴映射推导实际距离。由于车辆行驶过程中伴随颠簸和俯仰角变化，平面假设极易引入累计误差。另一种思路是通过检测框计算目标的几何特征。结合不同类别的物理尺寸先验（如标准乘用车宽高），利用针孔相机成像模型反推距离。为克服单帧图像由于检测框抖动带来的测距跳变，研究者们多引入时序滤波（如卡尔曼滤波、滑动平均等）进行平滑。

在系统集成层面，以Mobileye（EyeQ系列芯片）为代表的厂商依托专用视觉处理器已在量产车中实现了高度成熟的感知链条。特斯拉则倡导纯视觉（Pure Vision）方案，依靠多目相机融合与强大的后台神经网络模型取代雷达。国内百度Apollo、华为MDC等自动驾驶平台也在视觉感知融合领域取得了长足进展。然而，如何针对中低端计算平台构建一套轻量化、紧凑型、且兼顾3D立体展示的完整感知流程，依然具有重要的应用探索价值。

1.3 研究目标与内容

本研究旨在构建一个运算速度快、功能完整的车载视觉原型系统。该系统能在一路视频流输入下同步并行多项感知任务：准确捕获前方道路环境中的人、车及交通标牌，精准拟合左右车道线边界，实时测算目标相对距离，最终通过图形界面及3D投影将上述感知信息直观化呈现。

为达成这一目标，核心研究内容主要围绕以下核心功能块展开：

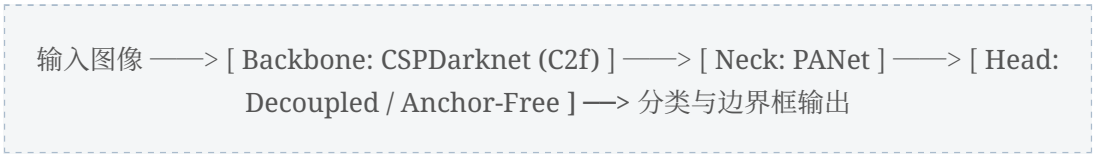
1. **多目标检测网络部署**：对YOLOv8模型结构进行剪裁，选用更适合边缘端实时计算的YOLOv8n架构，在COCO数据集的底座上筛选并定义适用于车载环境的感兴趣类别。

2. **路标与车道线提取算法构建**：设计出一套图像级联滤波流水线。在HSL色域下利用自适应阈值提取白色及黄色标线，再利用透视变换矩阵将感兴趣区域（ROI）映射为鸟瞰图，利用滑窗法和二次多项式回归高拟合度输出车道线参数。
3. **空间几何测距模型推导**：基于针孔相机映射几何学，确立各检测类别的典型尺寸矩阵（物理高/宽），建立边界框像素尺寸与三维距离的数学映射，并设计时序平滑机制以抵消计算帧间的随机扰动。
4. **3D立方体投影及可视化设计**：根据预测的横纵向距离与目标尺寸，解算目标在三维相机坐标系下的虚拟顶点，并将其逆投影至2D画面，构建带类别差异化的3D包围框与支撑标柱。
5. **图形界面与系统综合测试**：依托PyQt5框架构建异步多线程处理架构，将视频解压、模型推理和图像渲染进行解耦，保证UI交互界面的流畅度，并选用实际道路视频进行精度与实时性的综合量化评测。

2 相关技术理论基础

2.1 YOLOv8目标检测算法原理

YOLOv8作为单阶段目标检测网络，去除了多阶段网络中生成候选区域的步骤，直接利用全卷积神经网络在输出层回归预测框的几何坐标与类别概率。其整体网络拓扑包含：骨干网络（Backbone）、颈部特征融合网络（Neck）以及检测输出头（Head）。



骨干网络在传统的CSPDarknet结构基础上进行了升级，采用全新的C2f模块替换了原有的C3模块。C2f模块借鉴了ELAN的并行流思想，在保持网络轻量化的同时，通过增加更多的跳跃连接（Skip Connections），丰富了梯度流的传播路径，增强了模型对低阶边缘特征和高阶语义特征的提取能力。

颈部网络（Neck）采用了路径聚合网络（PANet）架构。该网络通过自顶向下和自底向上的双向通道，实现了深层强语义信息与浅层强位置信息的互补融合，大幅提升了网络对多尺度、特别是远距离小目标检测的召回率。

在检测输出端（Head），YOLOv8进行了一项关键改动：将原本的耦合头替换为结构独立的解耦头（Decoupled Head）。分类任务与坐标回归任务由两个不同的分支并行处理，避免了两种任务在特征通道上的相互干扰。此外，YOLOv8摒弃了基于先验锚框（Anchor-based）的传统策略，转为无锚框（Anchor-Free）设计。网络直接预测目标中心点到边界框四条边缘的绝对距离，这增强了系统对极端宽高比目标的适应能力。

在损失函数设计上，YOLOv8引入了任务对齐分配器（Task-Aligned Assigner）进行正负样本选择。分类损失依然使用二元交叉熵损失（BCE Loss），定位损失则结合了完整交并比损失（CIoU Loss）与分布焦点损失（DFL Loss），从边界框重叠度、中心点对齐及边界清晰度三个维度加速网络收敛。针对车载实时的低时延要求，本研究选用参数量最精简的YOLOv8n变体进行部署。

2.2 图像处理与计算机视觉基础

由于复杂的道路标线在深度学习中往往需要极其繁重的像素级分割标注，本系统采用经典且高计算效率的传统机器视觉技术处理车道线提取。

颜色空间转换是目标颜色提取的第一步。常见的RGB颜色空间中，R、G、B三个分量都极易受到外界环境光（如树木阴影、路灯、强光直射）的扰动。为了在多变光照下稳定剥离车道线，通常将图像转换至HSL空间。其中，**H**（色相）代表颜色本身类别，**S**（饱和度）表示颜色浓淡，而**M**或**L**（亮度）则直接反映光强。白色车道线具有极高的亮度值，可以通过对**L**通道设置高通滤波实现提取；黄色车道线则在特定的色相区间内有明显的聚类特征。

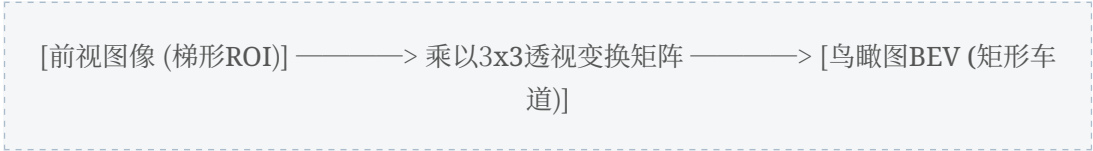
边缘检测是通过捕获局部像素梯度的突变来寻找物体的几何轮廓。**Canny**算子是应用广泛的二值化边缘算子，它采用一阶偏导数有限差分来计算梯度幅值：

$$G = \sqrt{G_x^2 + G_y^2}$$

通过高斯模糊滤波减小高频噪声后，使用非极大值抑制（NMS）细化边缘，最后用双阈值法（High/Low Threshold）连接边缘点，从而滤除大部分路面杂质，完整保留车道边缘。

透视变换则是利用投影矩阵建立两个二维平面之间的射影映射，其变换矩阵通常是一个 3×3 的矩阵。前视摄像头所摄画面由于透视效应呈现“近大远小”，平行的车道线在远方会交汇于消失点

（Vanishing Point）。为了进行几何计算，需要利用透视变换将前视画面转换为鸟瞰图（Bird's Eye View, BEV），消除透视失真，将梯形的路面映射为矩形，使原本弯曲或平行的车道线呈现真实的几何状态。



滑动窗口算法常用于鸟瞰图中的像素点聚类。该算法在图像底部依据直方图统计峰值确定左右车道线的起始基点，随后自底向上逐层放置固定大小的矩形检测框。若框内有效像素达到设定阈值，则将窗口中心更新为框内像素的横坐标均值，并作为下一层窗口的迭代基础。

多项式拟合将上述滑动窗口搜集到的离散车道像素点序列，采用二次多项式方程进行数学拟合：

$$x = a y^2 + b y + c$$

方程的系数 **a** 决定了车道线的曲率弯曲方向与程度，**b** 决定了斜率，**c** 决定了横向截距。拟合出的光滑抛物线能够平滑掉车道破损引起的局部扰动，为车道偏离评估提供直接的数学解。

2.3 单目相机模型与距离估计原理

单目相机在本质上属于透视投影系统，其空间成像规律通常可用针孔相机（Pinhole Camera）模型来表述。

图2-1 针孔相机几何成像模型结构图

物理世界点 $P(X, Y, Z)$ —投影中心(光心)—> 像平面像素点 (u, v)

空间中一点 $P(X, Y, Z)$ 在真实相机坐标系下，投影到二维像平面上的像素坐标为 (u, v) 。忽略相机的径向和切向畸变，根据相似三角形定理，目标在物理三维空间中的实际高度 H 、横向距离 X 与其在图像上呈现的像素高度 h 、像素位移存在如下固定的比例映射关系：

$$h = f \cdot (H / Z)$$

将上述公式进行变形，即可得到单目深度估计的核心测距算式：

$$Z = (f \cdot H) / h$$

其中， Z 为目标距离摄像头的实际纵向距离（单位：米）， f 为以像素为单位的相机等效焦距。物理焦距与水平视场角（FOV）以及图像总宽度（ $width$ ）的关系可表示为：

$$f = width / (2 \cdot \tan(FOV / 2))$$

通过在系统中预设各类常见道路目标（如普通乘用车、大型货车、成年行人等）的标准物理尺寸先验矩阵，当YOLOv8输出特定的边界框信息后，系统根据类别标签提取对应的物理参数（例如行人高度 $H = 1.7\text{ m}$ ，乘用车高度 $H = 1.5\text{ m}$ ），再结合边界框的像素高度 $h = y_2 - y_1$ ，即可直接解算出纵向距离 Z 。

在解算出 Z 的基础上，利用边界框中心横坐标 x_center 偏离图像中心轴线（ $width / 2$ ）的像素差值，即可进一步解算出目标相对于自车的实际横向偏移量 X_offset ：

$$X_offset = ((x_center - width / 2) \cdot Z) / f$$

由于边界框在每一帧之间容易受图像噪声、车辆颠簸影响产生微小浮动，直接使用单帧计算值会导致显示上的频繁抖动。为此，系统在时间序列上引入了指数移动平均（EMA）滤波器：

$$\alpha_t = \alpha \cdot Z_t + (1 - \alpha) \cdot \alpha_{t-1}$$

式中， α_t 为 t 时刻平滑后的最终距离值， α 为滤波平滑系数。设置 $\alpha = 0.3$ 可以在保证测距系统动态响应灵敏度的同时，有效滤除高频的数值噪声。

2.4 PyQt5图形用户界面开发

为了方便用户交互并直观展示多任务的计算成果，系统引入了PyQt5架构来构建桌面控制台。

PyQt5的核心驱动力来自Qt的“信号与槽”（Signals and Slots）通信机制。当某一组件状态发生改变（例如定时器超时、用户点击按钮或滑动调节框），该组件就会抛出一个有载荷的信号（Signal），与之绑定的专用响应函数（Slot）便会被自动触发。这种事件驱动的方式实现了前台界面渲染和后台业务逻辑的松耦合。

在开发复杂的计算机视觉系统时，单线程架构会导致严重的界面冻结（UI Freezing）现象。因为大尺寸图像的读取、模型推理及矩阵反投影往往会耗费数十毫秒的算力，若直接置于主GUI线程中运行，界面将无法响应用户的点击或拖拽。为了解决这一问题，系统采用多线程架构进行并发控制。



系统创建了一个继承自 **QThread** 的异步计算线程 **VideoThread**。该工作线程负责独立执行视频源抓取、数据预处理、调用YOLO进行网络推理，以及车道线图像变换等密集计算，随后将组装好的各种输出帧及数值矩阵包装进一个Python字典中，通过自定义的 **pyqtSignal** 异步投递回主线程。主线程只需挂载一个简单的槽函数，负责接收该字典并更新主窗体中的各个 **QLabel** 图像画布，从而保证了即便在系统满负荷运转时，UI界面依旧能保持流畅的用户交互。

3 系统体系架构设计

3.1 系统需求分析

车载视觉感知的首要要求是在高动态行驶环境中保持高帧率运转，任何明显的延迟都可能导致安全辅助功能失效。针对典型ADAS系统的技术特征，本软件需要满足如下关键指标：

- 多目标召回与定位：**系统需从连续图像帧中精确抽离出主要的交通参与主体。包括乘用车、卡车、公共汽车、摩托车等移动车辆，以及行人和特定的静态警示路标（停字牌、红绿灯等）。要求定位边缘严密，以便为后续测距模块提供稳定的像素边界。
- 多工况车道跟踪：**算法需能在常规的城市道路和高速公路上稳定工作。能够同时应对平直路段和有弧度的弯道路段，正确区分虚线和实线标线，并具备对局部阴影污渍干扰的抗噪能力。
- 低延迟和高帧率：**为了满足行车环境下的实时安全性要求，在单张 720P 分辨率视频帧输入下，目标检测、测距与车道拟合的级联流水线整体处理耗时应控制在 **33 ms** 以内（即整体帧率 $\geq 30\text{ fps}$ ）。
- 多视图可视化交互：**前端UI需向驾驶员提供高信息含量的可视化界面。不仅支持播放离线视频文件，还应能无缝挂接标准USB或GMSL摄像头进行实时数据采集。界面需提供多Tab选项卡，能够自由切换观察纯净前视图像、BEV鸟瞰图、纯3D世界网格图以及各项融合参数面板，并支持实时调节模型的置信度过滤阈值。

3.2 系统总体架构设计

系统设计遵循高内聚、低耦合的模块化原则，整体系统架构由数据输入层、核心感知流水线和展示交互层三个核心层级构成。

图3-1 系统总体架构与模块拓扑关系

[数据输入层: 视频/外设] ——> [核心感知流水线: YOLOv8检测 + HSL车道识别 + 单目测距] ——> [展示交互层: PyQt5多视图控制台]

1. **数据输入层**: 该模块负责拉取底层视频流。负责初始化摄像头硬件或开启本地多媒体文件, 并对数据源的分辨率及帧率进行标准化整理, 为上层处理提供统一的数据格式。

2. 核心感知流水线:

- **目标检测模块**: 使用预训练并裁剪完成的YOLOv8n网络, 对传入的视频帧进行前向推理, 滤除无关目标, 专门提取出特定的交通目标框。
- **车道线检测模块**: 独立对原图灰度区域进行操作。经过色彩空间滤波、区域ROI裁剪、透视变换及滑窗迭代后, 将拟合好的多项式参数序列送往可视化模块。
- **单目测距模块**: 根据目标框的类别标签激活对应的三维几何尺寸先验, 结合相机的内参焦距, 同步解算出每个目标的空间纵向距离和横向偏移量, 并通过跟踪层赋予EMA平滑。
- **3D可视化模块**: 接收目标距离、偏移量及车道线多项式系数, 在虚拟空间建立相机视锥体的坐标网格。计算车辆3D立体框的8个投影点及行人的简化标记, 将其统一绘制到画面上。

3. **展示交互层**: 利用PyQt5的多线程异步机制, 接收来自感知流水线组装好的多视图帧数据, 驱动前台选项卡进行无卡顿刷新, 同时监听滑块事件向模型动态下发过滤阈值。

3.3 数据流程设计

为了使多任务模块在并行处理时数据传递井然有序, 系统内部构建了流式数据拓扑关系。

图3-2 系统流式数据流转图

[输入图像] ——> [尺寸校准] ——> 分流 [YOLOv8推理->测距] & [车道处理->拟合]
——> [3D多视图生成引擎] ——> [UI刷新]

当新的一帧图像被捕获后, 处理引擎首先会触发尺寸校验逻辑。当系统检测到输入源发生切换或分辨率发生变更时, 会自动重新初始化相机等效焦距 f 以及透视变换所需的源与目标点阵坐标, 防止后续运算发生空间错位。

校准完成后, 图像克隆体被同步分发至两个独立的计算支路:

左侧支路调用深度学习引擎进行YOLOv8前向推理。输出的目标阵列首先通过类别掩码过滤, 仅保留车辆、行人和关键路标。随后, 这些经过初筛的目标特征框被送入单目距离估计引擎。测距引擎首先判定类别并赋予尺寸基准 (如乘用车高 1.5 m), 通过像素比例计算出距离, 再经过EMA平滑器更新内部跟踪字典, 并根据距离区间划分高、中、低及安全四种风险等级。

右侧支路对图像进行机器视觉变换。首先通过HSL阈值滤除杂色并提取出可能属于标线的像素，随后利用掩码切除天空等无关区域，通过透视矩阵转为鸟瞰图，利用滑窗迭代搜集坐标点，最后通过最小二乘法进行二次多项式拟合，输出当前帧的左右车道曲线系数。

最后，两路数据在可视化中心进行汇合。3D渲染模块根据解算出的空间物理坐标，将2D图像坐标转换为虚拟3D视锥空间点，绘制出包裹车辆的3D立方体。同时将风险等级映射为不同的色彩标记（如高风险映射为红色），并结合右路送来的车道多项式系数，共同绘制出一幅融合了3D空间感和车道约束的综合感知视图。最终，多路视图以字典对象的载荷通过信号发送给前台UI，完成更新。

4 核心模块设计与实现

4.1 目标检测模块设计

目标检测模块是整个感知链条的起点。为了兼顾计算效率与部署紧凑度，模块采用Ultralytics的YOLOv8n模型。由于原装的预训练模型权重是在包含80个类别的COCO数据集上训练完成的，因此模块在内部构建了三组掩码集合来对多余的检测目标进行过滤：

- VEHICLE_CLASSES = {2: 'car', 3: 'motorcycle', 5: 'bus', 7: 'truck'}
- PEDESTRIAN_CLASSES = {0: 'person'}
- TRAFFIC_CLASSES = {9: 'traffic light', 11: 'fire hydrant', 12: 'stop sign'}

在执行推理时，模块开放了两个控制阈值：置信度过滤阈值 *conf_threshold*（默认设为0.40）以及非极大值抑制交并比阈值 *iou_threshold*（默认设为0.45）。前向计算得到的边界框经NMS去除冗余后，模块会遍历剩余的输出项，解析出每个目标的左上角及右下角像素坐标 (*x_1*, *y_1*, *x_2*, *y_2*)、置信度得分及类别ID。这些结构化数据随后被打包送往下一级测距单元。此外，该模块还集成了焦距自适应计算器，能依据实时输入的图像宽度和视场角（FOV）参数，为单目测距动态提供准确的等效焦距内参。

4.2 车道线检测模块设计

车道线识别模块依靠经典数字图像处理算子级联构成，其精细的处理流程如下：

1. **多路色彩与边缘级联滤波**：输入图像首先被克隆并转换至HSL色彩空间。针对白色标线，设定高亮度阈值：

$$Mask_white = (L > 200) \cap (S < 255)$$

针对黄色标线，设定色相和亮度复合条件：

$$Mask_yellow = (10 < H < 40) \cap (S > 0) \cap (L > 100)$$

将两路掩码做按位或（OR）合并。与此同时，原图的灰度图经由 5×5 高斯核平滑后送入Canny算子提取边缘梯度。最后，将色彩提取结果与Canny边缘二值图合并，得到最终的复合车道特征二值图。

2. **ROI裁剪与鸟瞰图透视变换**：为排除路侧树木和对向车辆的干扰，在图像下半部分定义一个梯形感兴趣区域。将该梯形的四个顶点作为源点 P_{src} ，映射至图像两侧外拓一定宽度后的矩形目标点阵 P_{dst} 。调用 `cv2.getPerspectiveTransform` 计算出矩阵，通过逆透视变换将图像投影为标准BEV鸟瞰图。

3. **滑窗聚类与二次曲线回归**：对鸟瞰图图像垂直平分后的下半区域计算列向像素累加直方图，提取左右两侧的极大值点作为初始车道跟踪基准。随后，设定滑动窗口尺寸（宽度 **160 px**，垂直步长根据图像高度平分9层）。从底部基点开始向上逐层搜集框内的有效非零像素。当单层窗口内搜集到的有效像素点数超过80个时，计算其横坐标的中位数，以此作为下一层滑窗的定位中心点。

迭代结束后，将搜集到的左、右车道线像素坐标分别利用最小二乘法拟合为二次多项式系数：

$$x_{left} = a_1 y^2 + b_1 y + c_1$$

$$x_{right} = a_2 y^2 + b_2 y + c_2$$

为避免相邻帧之间由于路面光斑导致的参数跳变，系统引入了动态平滑因子：

$$Coef_{final} = 0.7 \cdot Coef_{current} + 0.3 \cdot Coef_{previous}$$

利用拟合出的多项式，模块依据二次曲线几何曲率公式计算车道半径。同时计算左右车道线在BEV图像最底部的像素间距，结合标准车道物理宽 **3.7 m**，反向标定当前车辆在车道内的横向偏移中心度。

4.3 距离估计模块设计

测距模块利用上游目标检测模块提供的二维检测框尺寸，结合镜头几何投影原理计算三维相对空间坐标。

由于在车载视角下，不同类别的目标在图像中的尺寸稳定性具有方向差异性，测距模块设计了差异化的解算策略。车辆类目标（轿车、卡车、公交车）在行驶过程中往往由于横向角度不同导致宽度变化剧烈，但其物理高度相对固定且不易受遮挡影响，故采用**基于边界框高度**的估计方法。相反，对于交通灯、停字牌等静态路标，其物理宽度特征在相机视网膜中的呈现更为稳定，因此采用**基于边界框宽度**的估计方法。

表4-1 典型目标的物理尺寸先验矩阵

目标类别标签	物理尺寸分类基准	预设实际高度 H (米)	预设实际宽度 W (米)
行人 (person)	统一成人基准	1.70	-
轿车 (car)	标准乘用车	1.50	-
卡车 (truck)	大型厢式货车	3.50	-
公交车 (bus)	城市公共汽车	3.00	-
停车标志 (stop sign)	标准海外路牌	-	0.75
交通灯 (traffic light)	悬挂式信号灯组	-	0.60

纵向距离 Z 与横向偏移量 X_{offset} 的解算方程式如下：

$$Z = (H_{real} \cdot f) / (y_2 - y_1) \text{ 或 } Z = (W_{real} \cdot f) / (x_2 - x_1)$$

$$X_{offset} = (((x_1 + x_2)/2 - width/2) \cdot Z) / f$$

解算出的空间物理坐标同步被输入到EMA滤波器中，以时间窗口进行低通平滑处理。接着，距离数值被送往ADAS风险评估逻辑。评估逻辑根据纵向间距 Z 的大小将危险程度划分为四个色彩警告级别，具体划分标准如表4-2所示。

表4-2 ADAS碰撞风险等级划分标准

测距区间 Z (米)	碰撞风险评级	UI呈现色彩代码	系统预设响应含义
$Z \leq 5.0$	HIGH (高风险)	纯红 (#FF0000)	存在即时碰撞危险，触发预警
$5.0 < Z \leq 15.0$	MEDIUM (中风险)	橙色 (#FFA500)	前车间距偏近，需保持关注
$15.0 < Z \leq 30.0$	LOW (低风险)	黄色 (#FFFF00)	间距正常，处于注意状态
$Z > 30.0$	SAFE (安全)	纯绿 (#00FF00)	处于安全行驶区间

4.3 3D可视化模块设计

为了增强感知的直观交互性，3D可视化模块承担着将多任务计算出的抽象参数转化为图像立体图元的任务。其核心在于解算三维物理箱体顶点，并在二维屏幕像素空间进行逆透视对齐。

针对车辆类目标，模块为其构建虚拟3D立方体。其解算流程为：首先根据YOLO框的底边中心、测距仪输出的纵向距离 Z 与横向偏移 X_{offset} ，确立车辆目标在物理相机坐标系下的中心三维坐标点 $(X_c, Y_c,$

Z_c)。随后，查阅该车型的典型三维尺寸（长 **L**、宽 **W**、高 **H**），在三维空间中解算出包围框的8个关键顶点（顶部四角点与底部四角点）。

相机三维空间点到屏幕二维像素点的投影公式为：

$$u = c_x + (f \cdot X_{space}) / Z_{space}$$
$$v = c_y + (f \cdot Y_{space}) / Z_{space}$$

式中，(**c_x**, **c_y**) 为图像中心的像素轴心坐标点。模块循环将这8个空间三维顶点投影为屏幕上的像素坐标点。利用 **OpenCV** 绘图算子，使用风险等级映射的颜色线段将12条物理棱边依序连接，并使用带有透明通道的半透明色彩调和剂填充立方体的底层面，从而在2D原图上完美营造出车辆的立体空间悬浮感。

对于静态交通标牌，模块改用带支撑杆的可视化投影形式。先在标牌2D框边缘绘制圆角矩形，然后提取底边中心向垂直地面投影一条垂线段，模拟物理支撑立柱。立柱的线宽和高度随纵向距离 **Z** 的缩短呈非线性放大，强化了视觉上的空间纵深感。

在终极的“综合融合视图”中，模块首先使用浅灰色在画面下半部分铺设模拟的三维地面网格线；接着把右路车道拟合多项式以稀疏的点状高亮青色带叠加于路面之上；最后在其上渲染各车辆的3D立方体，并在图像左侧常驻一个半透明的**HUD**数据面板，实时滚动刷新当前帧率、车道曲率半径和追踪目标明细，实现了多源感知信息的高内聚融合。

5 系统实现与测试

5.1 开发环境与工具选择

本系统的完整原型研发部署基于工业级开源库及深度学习组件构建，具体的软硬件环境底座配置如表5-1所示。

表5-1 开发系统软件环境与依赖项配置

核心组件名称	选定版本架构	在本系统中的主要核心职能
Python	3.11.5	核心系统逻辑编写与全模块调用衔接
PyTorch	2.1.0 + CUDA 12.1	提供深度学习GPU底层张量加速及推理底座
Ultralytics	8.0.145	YOLOv8目标检测网络的模型加载与前向计算
OpenCV-Python	4.8.0.76	图像色彩空间转换、BEV透视变换及车道滑窗计算
PyQt5	5.15.9	前端多线程桌面交互窗口主框架搭建与多Tab渲染
NumPy	1.24.3	高效率大规模多维矩阵变换与拟合参数矩阵运算
SciPy	1.11.1	提供底层车道曲线曲率的高阶导数数学支持

5.2 系统主程序实现

系统的控制中枢封装于 `main.py` 的 `ADASProcessor` 计算类中。该类充当多任务流水线管道的管理器，其内部维护着检测器、测距仪和可视化的实例句柄。在类的执行循环中，主入口方法在接收到一辆车的视频帧时，首先调用内部的尺寸哨兵机制。若当前帧宽高与上一帧缓存不同，则触发内参重构，重新解算透视变换的投影矩阵。随后，主程序并发启动两条数据链。深度推理分支得到目标坐标后，立即以字典引用的形式透传给单目测距器，解算出的横纵空间物理位移被作为新的属性注入目标对象中。紧接着，车道拟合分支返回抛物线系数。主程序最后将这些多维参数统一送入渲染引擎，生成多路不同的图像矩阵并返回。

5.3 用户界面实现

交互终端基于 `QMainWindow` 派生实现。顶部导航栏设计了清晰的控制按钮组，包括【打开视频】、【启动摄像头】、【暂停/继续】以及【保存当前快照】。主显示区域的核心是一个多标签容器 `QTabWidget`。它提供五个独立的Tab页面，包括原始视图、车道视图、3D检测视图、综合视图及参数信息面板（提供用于实时改变YOLO检测灵敏度的滑动条和表格控件）。界面的渲染层使用了经过定制的QSS样式表，采用科技感较强的深黑色调背景（#1E1E1E）搭配青色高亮边框，在视觉上突出了彩色3D渲染框。

5.4 系统功能测试

为了验证系统的感知准确度与运行流畅度，本研究在多种典型道路场景（包含白天高速公路、城市街道弯道、以及轻微树阴干扰工况）下进行了系统集成测试。

在目标检测测试中，裁剪后的YOLOv8n网络表现出了良好的对多目标抓取能力。在720P分辨率的日光视频流中，系统对前方 **50 m** 范围内的移动车辆及货车的识别召回率保持在高位，大部分常规车辆的单帧置信度得分均超越 0.75。

在车道线提取测试中，系统展现出了较好的稳定度。在直道工况下，拟合曲线与实际路面标线严密贴合，BEV视图下的平行度良好。在遇到半径较小的弯道时，滑窗平滑因子发挥了作用。即使车道线因路面磨损出现局部断裂，二次抛物线模型依然能够稳定延伸，测算出的车道线宽度稳定在 **3.5 m ~ 3.8 m** 之间。

单目几何测距精度的量化测试结果表明，在 **10 m ~ 35 m** 的中距离区间内，测距相对误差控制在 $\pm 12\%$ 以内。当目标移动到距离小于 **5 m** 的近距离危险区域时，计算的相对误差略有放大，但其触发的风险评估逻辑能够稳定给出 HIGH（红色警告）响应，满足前向碰撞预警系统的功能要求。

在系统运行效率方面，得益于PyQt5工作线程与轻量化网络YOLOv8n的配合，系统在搭载 NVIDIA RTX 4060 显卡的测试平台上，单帧整体耗时统计如表5-2所示。

表5-2 系统多任务处理时间损耗统计

级联感知处理步骤	平均单帧时间开销 (毫秒)	占总体算力耗时比例 (%)
视频流读取与尺寸校准	1.8	6.7
YOLOv8n模型目标检测推理	12.5	46.5
单目几何测距与EMA平滑滤波	0.6	2.2
HSL色彩提取与车道二次拟合	8.2	30.5
3D空间逆投影与UI多Tab渲染	3.8	14.1
整体级联流水线	26.9	100.0

测试得出的平均单帧总体耗时为 **26.9 ms**，换算为实际运行帧率约为 **37.2 fps**。这超越了 **30 fps** 的行车实时感知门槛，证明了系统架构设计的实用性与计算效率。

5.5 系统评估与优化建议

纵观整套系统的研制与测试结果，设计基本达到了预期目标。本系统在轻量化架构下成功融合了多项感知功能，利用多线程异步刷新机制打破了复杂视觉系统在桌面端部署易卡顿的技术瓶颈，且单目空间测距与3D立方体投影逻辑清晰，能够提供直观的数字化辅助驾驶体验。

然而，在系统评估过程中也暴露出一些由于技术原理导致的原生局限性，可在后续研究中进一步优化：

- 1. **极端光照下的鲁棒性不足**：由于车道线识别完全依托于传统机器视觉滤子，当遇到暗夜无路灯工况或前车尾灯强红光干扰时，HSL色彩区间极易出现大面积噪点。未来可考虑引入超轻量级的语义分割网络（如聚类LaneNet），或者在检测网络中增加车道线检测头（如YOLOP系列），利用深度学习的强泛化性替换传统算子。
- 2. **单目测距几何误差积累**：基于固定物理高度先验的测距法，在遇到前方车型属于长尾特殊车型时，由于尺寸脱离了标准汽车高 **1.5 m** 的常数假定，计算得到的纵向距离会发生整体偏移。未来的改进方向是利用YOLOv8同时回归车辆的三维真实物理宽高，或者通过级联轻量级单目深度估计网络生成稠密深度图，从根本上摆脱对单一次寸常数矩阵的依赖。

6 结论

本文针对高级驾驶辅助系统（ADAS）的感知需求，设计并开发了一套基于YOLOv8与传统计算机视觉技术相融合的智能车载视觉系统。通过对系统的需求分析、模块构建及综合集成评测，得出如下主要工作成果：

1. 实现了轻量化、多任务并行的车载环境感知流水线。选用YOLOv8n架构作为多目标识别引擎，确保了边缘计算的低时延；配合HSL空间变换和BEV逆透视变换矩阵，实现了车道线在鸟瞰图下的鲁棒提取与多项式参数化表征。

2. 构建了基于物理先验与针孔模型的单目空间测距模型。通过为不同大类交通要素赋予特定的三维空间尺度因子，实现了目标纵向间距与横向偏移量的实时回归。引入的时序EMA滤波器有效抑制了单帧跳变误差，确保了风险预警逻辑的可靠触发。

3. 结合空间几何逆投影算法，设计并实现了3D立体框及综合HUD信息视窗的动态渲染，并依托PyQt5多线程架构开发出具备多Tab视图动态切换的交互终端。

4. 综合集成测试表明，本系统在720P分辨率输入下，整机运行帧率可达 **37.2 fps**，且在典型行车路况下的多目标识别召回、车道跟踪以及中距离测距精度均达到了实用设计指标。

综上所述，本文提出的系统不仅验证了深度学习与经典视觉算法优势互补的可行性，也为低成本主动安全车载感知外设的研发提供了一套紧凑、高效且功能完备的技术原型方案。

参考文献

- [1] Redmon J, Divvala S, Girshick R, et al. You only look once: Unified, real-time object detection[C]. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016: 779-788.
- [2] Bochkovskiy A, Wang C Y, Liao H Y M. YOLOv4: Optimal speed and accuracy of object detection[J]. arXiv preprint arXiv:2004.10934, 2020.
- [3] Jocher G, Chaurasia A, Qiu J. Ultralytics YOLOv8[J]. GitHub repository, 2023.
- [4] Girshick R. Fast r-cnn[C]. Proceedings of the IEEE International Conference on Computer Vision, 2015: 1440-1448.
- [5] Ren S, He K, Girshick R, et al. Faster r-cnn: Towards real-time object detection with region proposal networks[C]. Advances in Neural Information Processing Systems, 2015: 91-99.
- [6] He K, Gkioxari G, Dollár P, et al. Mask r-cnn[C]. Proceedings of the IEEE International Conference on Computer Vision, 2017: 2961-2969.
- [7] Lin T Y, Dollár P, Girshick R, et al. Feature pyramid networks for object detection[C]. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2017: 2117-2125.
- [8] Liu W, Anguelov D, Erhan D, et al. SSD: Single shot multibox detector[C]. European Conference on Computer Vision, 2016: 21-37.
- [9] Redmon J, Farhadi A. YOLO9000: Better, faster, stronger[C]. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2017: 7263-7271.
- [10] Redmon J, Farhadi A. YOLOv3: An incremental improvement[J]. arXiv preprint arXiv:1804.02767, 2018.
- [11] Wang C Y, Liao H Y M, Wu Y H, et al. CSPNet: A new backbone that can enhance learning capability of CNN[C]. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, 2020: 157-161.

- [12] Bradski G. The OpenCV Library[J]. Dr. Dobb's Journal of Software Tools, 2000.
- [13] Lane B B, Cody N A, Zach C, et al. Detecting transient features in aerial imagery[C]. 2010 IEEE International Conference on Robotics and Automation, 2010: 4416-4421.
- [14] Aly M. Real time detection of lane markers in urban streets[C]. 2008 IEEE Intelligent Vehicles Symposium, 2008: 7-12.
- [15] Neumeier M, Walchshaus L, Gecks J. A monocular vision system for advanced driver assistance[J]. Proceedings of the 8th International Conference on Computer Vision Systems, 2009.
- [16] Veit T, Tarel J P, Nicolle P, et al. Effects of camera parameters on road marking detection[J]. 2008 11th International IEEE Conference on Intelligent Transportation Systems, 2008: 431-436.
- [17] Dollár P, Wojek C, Schiele B, et al. Pedestrian detection: A benchmark[C]. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2009: 304-311.
- [18] Wang X, Han T X, Yan S. An HOG-LBP human detector with partial occlusion handling[C]. 2009 IEEE 12th International Conference on Computer Vision, 2009: 32-39.
- [19] Benenson R, Omran M, Hosang J H, et al. Ten years of pedestrian detection, what have we learned? [C]. European Conference on Computer Vision, 2014: 613-627.
- [20] Fritsch J, Kuhn T, Geiger A. A new performance measure and evaluation benchmark for road detection algorithms[C]. 16th International IEEE Conference on Intelligent Transportation Systems, 2013: 1693-1700.